🧑‍💻 DEVELOPER GUIDE - ECOHUB QR SCANNER

Dành cho developers muốn customize hoặc extend hệ thống

🚀 SETUP DEVELOPMENT

1. Clone/Download project

```
cd eco_hub_demo
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Run development server

```
python app.py
```

Output:

```
============================================================
  ECOHUB QR SCANNER - READY
============================================================
  URL: http://127.0.0.1:5000
  Camera Status: NOT STARTED (manual start required)
  S3 Status: CONFIGURED
============================================================

 * Running on http://127.0.0.1:5000
 * Debug mode: on
```

4. Hot reload

Flask debug mode enabled → Code changes auto-reload

📂 CODE STRUCTURE

Main Application (`app.py`)

```
# Global state
app_state = {
    "current_order_code": None,
    "is_recording": False,
    ...
}

# Upload queue system
upload_queue = Queue()
upload_status_dict = {}
upload_worker()  # Background thread

# Camera management
camera_managers = []  # CameraManager instances
ai_scanners = []      # AIBarcodeScanner instances

# Flask routes
@app.route("/dashboard")
@app.route("/start_recording", methods=["POST"])
...
```

Camera Module

`camera/camera_manager.py`

- Camera stream management
- Frame reading loop (10ms interval)
- Queue management (AI + Recorder)
- MJPEG encoding
- RTSP buffer optimization

Key methods:

```
class CameraManager:
    def start()                     # Start camera thread
    def stop()                      # Stop camera thread
    def get_latest_frame()          # Get frame for display
```

```
    def get_frame_from_queue()       # Get frame for AI
    def _loop()                      # Main camera loop
```

`camera/ai_scanner.py`

- QR/Barcode detection with ZXing-CPP
- Timestamp-based FPS control (5 FPS)
- Cooldown mechanism (5 minutes)
- Memory leak prevention (periodic cleanup)

Key methods:

```
class AIBarcodeScanner:
    def start()                      # Start AI thread
    def stop()                       # Stop AI thread
    def pause()                      # Pause during recording
    def resume()                     # Resume after recording
    def reset()                      # Clear queue for fresh frames
    def _loop()                      # Main AI loop
```

`camera/recorder.py`

- Async video recording
- Separate writer thread
- 90-frame buffer (non-blocking)
- VideoWriter management

Key methods:

```
class VideoRecorder:
    def start(path, frame_size, fps) # Start recording
    def write_frame(frame)           # Write frame (non-blocking)
    def stop()                       # Stop recording
    def _write_loop()                # Writer thread loop
```

`camera/camera_stream.py`

- MJPEG stream generation
- Bounding box overlay
- Real-time FPS calculation

Key function:

```
def generate_mjpeg(camera_manager, ai_scanner):
    # Yield MJPEG frames
    while True:
        frame = camera_manager.get_latest_frame()
        # Draw bounding boxes
        # Encode to JPEG
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + jpeg + b'\r\n')
```

Services Module

`services/s3_service.py`

- S3 operations (upload, list, delete, download)
- Multipart upload (50MB threshold, 10MB chunks)
- Presigned URL generation
- Retry mechanism

Key methods:

```
class S3Service:
    def upload_file(filepath, object_key)        # Upload
    def list_videos()                            # List S3 objects
    def delete_video(object_key)                 # Delete
    def generate_presigned_url(object_key)       # Download URL
```

`services/config_encryption.py`

- Fernet symmetric encryption
- Key generation (PBKDF2-HMAC-SHA256)
- Encrypt/decrypt credentials

Key methods:

```
class ConfigEncryption:
    def encrypt(plaintext)                       # Encrypt string
    def decrypt(encrypted)                       # Decrypt string
```

`services/storage_service.py`

- Local file management

- Recording filename generation
- Mock implementation

`services/order_service.py`

- Order info retrieval
- Mock implementation (API integration point)

🔧 CUSTOMIZATION

1. Change Cooldown Time

File: `camera/ai_scanner.py`

```
# Line ~16
COOLDOWN_SECONDS = 5 * 60  # Change to 2 * 60 (2 minutes)
```

---

2. Change AI FPS

File: `camera/ai_scanner.py`

```
# Line ~15
TARGET_INTERVAL = 0.2  # 5 FPS → Change to 0.1 (10 FPS)
```

---

3. Change Recording FPS

File: `app.py` → `start_recording()`

```
# Line ~1205
recorder.start(video_path, frame_size=frame_size, fps=15.0)
# Change to fps=20.0
```

---

4. Change Max Cameras

File: `app.py`

```
# Line ~53
MAX_CAMERAS = 2  # Change to 3 or 4
```

---

5. Change Upload Threshold

File: `services/s3_service.py`

```
# Line ~22-23
MULTIPART_THRESHOLD = 50 * 1024 * 1024  # 50 MB → 100 MB
MULTIPART_CHUNKSIZE = 10 * 1024 * 1024  # 10 MB → 20 MB
```

---

6. Add Custom Order API

File: `services/order_service.py`

```
def get_order_info(order_code: str):
    # Current: Mock data

    # Replace with real API:
    import requests
    response = requests.get(f"https://api.example.com/orders/{order_code}")
    return response.json()
```

---

7. Add Custom Video Processing

File: `app.py` → `stop_recording()`

```
# After recording stops, before upload

# Add watermark
import cv2
video = cv2.VideoCapture(video_path)
# ... process frames ...

# Add audio
import moviepy
# ... add audio track ...
```

```
# Compress video
import ffmpeg
ffmpeg.input(video_path).output(compressed_path, vcodec='h264').run()
```

🧪 TESTING

Unit Tests (TODO)

```
# Install pytest
pip install pytest

# Run tests
pytest tests/
```

Manual Testing

Test Camera:

```
from camera.camera_manager import CameraManager, SOURCE_USB

mgr = CameraManager(camera_index=0, source_type=SOURCE_USB)
mgr.start()
frame = mgr.get_latest_frame()
print(f"Frame shape: {frame.shape}")
mgr.stop()
```

Test AI Scanner:

```
from camera.ai_scanner import AIBarcodeScanner

def callback(code, bbox):
    print(f"Detected: {code}")

scanner = AIBarcodeScanner(camera_manager=mgr, on_code_detected=callback)
scanner.start()
# ... wait for QR detection ...
scanner.stop()
```

Test S3:

```
from services.s3_service import S3Service, S3Config

config = S3Config(
    endpoint="https://...",
```

```
    region="ap-southeast-1",
    bucket="test-bucket",
    access_key="AKIA...",
    secret_key="wJalr..."
)

service = S3Service(config)
result = service.upload_file("test.mp4", "test_video.mp4")
print(f"Upload: {result}")
```

🐛 DEBUGGING

Enable Debug Logs

Camera Debug:

```
# camera/camera_manager.py (line ~150)
if frame_count % 150 == 0:  # Change to % 30 (more frequent)
    print(f"[CAMERA DEBUG] ...")
```

AI Debug:

```
# camera/ai_scanner.py (line ~145)
print(f"[AI DEBUG] fps={fps:.1f}, t_total={t_total:.3f}s, ...")
# Always on
```

Upload Debug:

```
# app.py → upload_worker()
print(f"[UPLOAD WORKER] Uploading {task.filename}...")
print(f"[UPLOAD WORKER] Result: {result}")
# Always on
```

Visual Studio Code

`.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Flask",
      "type": "debugpy",
```

```
      "request": "launch",
      "module": "flask",
      "env": {
        "FLASK_APP": "app.py",
        "FLASK_DEBUG": "1"
      },
      "args": ["run", "--host=127.0.0.1", "--port=5000"],
      "jinja": true
    }
  ]
}
```

🔐 SECURITY CONSIDERATIONS

Encryption

```
# config_encryption.py uses Fernet (symmetric)
# Key derivation: PBKDF2-HMAC-SHA256, 100,000 iterations
# Key storage: config.key (44 bytes)

# NEVER commit config.key to git!
# Add to .gitignore:
config.key
config.json  # Contains encrypted credentials
```

Session Management

```
# app.py
app.secret_key = "ecohub-demo-secret"  # Change in production!

# Better:
import secrets
app.secret_key = secrets.token_hex(32)
```

S3 Permissions

Minimum IAM Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
```

```
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::your-bucket/*",
        "arn:aws:s3:::your-bucket"
      ]
    }
  ]
}
```

🚀 DEPLOYMENT

Production Checklist

- [ ] Change `app.secret_key`
- [ ] Set `console=False` in `build.spec` (no console window)
- [ ] Add icon: `icon='icon.ico'` in `build.spec`
- [ ] Test on clean Windows machine
- [ ] Test camera (USB + RTSP)
- [ ] Test S3 upload/download/delete
- [ ] Test full workflow (QR → Record → Upload)
- [ ] Test installer (install/uninstall)

Build for Production

```
# 1. Update version
# build.spec: name='EcoHub_QR_Scanner_v1.0.1'
# setup.iss: #define MyAppVersion "1.0.1"

# 2. Clean build
rmdir /s /q dist build

# 3. Build EXE
pyinstaller build.spec --clean

# 4. Test EXE
dist\EcoHub_QR_Scanner\EcoHub_QR_Scanner.exe

# 5. Build Installer
"C:\Program Files (x86)\Inno Setup 6\ISCC.exe" setup.iss
```

```
# 6. Test Installer
installer_output\EcoHub_QR_Scanner_Setup_v1.0.1.exe
```

📊 PERFORMANCE TUNING

1. Optimize AI FPS

Current: 5 FPS (0.2s interval)

Higher FPS (faster detection):

```
TARGET_INTERVAL = 0.1  # 10 FPS
```

Lower FPS (less CPU):

```
TARGET_INTERVAL = 0.5  # 2 FPS
```

---

2. Optimize Frame Resize

Current: 480px width

Smaller (faster but less accurate):

```
new_width = 320  # 320px
```

Larger (slower but more accurate):

```
new_width = 640  # 640px
```

---

3. Optimize Recording Buffer

Current: 90 frames (6 seconds @ 15 FPS)

Smaller (less memory):

```
_frame_queue = queue.Queue(maxsize=45)  # 3 seconds
```

Larger (smoother recording):

```
_frame_queue = queue.Queue(maxsize=150)  # 10 seconds
```

---

4. Optimize Upload

Parallel uploads:

```
# Currently: 1 upload worker thread

# Add more workers:
for i in range(3):  # 3 workers
    thread = threading.Thread(target=upload_worker, daemon=True)
    thread.start()
```

🔌 INTEGRATION POINTS

1. Order API Integration

Current: Mock data in `services/order_service.py`

Replace with real API:

```
import requests

def get_order_info(order_code: str):
    try:
        response = requests.get(
            f"https://api.ecohub.vn/orders/{order_code}",
            headers={"Authorization": "Bearer <token>"},
            timeout=5
        )
        response.raise_for_status()
        return response.json()
    except Exception as e:
        print(f"[ORDER API] Error: {e}")
        return None
```

---

2. Webhook on Upload Complete

File: `app.py` → `upload_worker()`

```
# After successful upload
if result.startswith("s3://"):
    # Send webhook
    import requests
    requests.post("https://api.ecohub.vn/webhooks/video-uploaded", json={
        "order_code": task.order_code,
        "filename": task.filename,
        "s3_url": result,
        "timestamp": datetime.now(GMT7).isoformat()
    })
```

---

3. Database Integration

Install SQLAlchemy:

```
pip install sqlalchemy
```

Create models:

```
# models.py
from sqlalchemy import create_engine, Column, String, DateTime
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class VideoRecord(Base):
    __tablename__ = 'videos'

    id = Column(String, primary_key=True)
    order_code = Column(String)
    filename = Column(String)
    s3_url = Column(String)
    created_at = Column(DateTime)
    status = Column(String)
```

---

4. Custom QR Actions

File: `app.py` → `on_code_detected()`

```
def on_code_detected(code, bbox):
    # Current: Auto-record on QR

    # Custom actions based on QR prefix:
    if code.startswith("ORDER_"):
        # Regular order → Auto-record
        _trigger_auto_recording(code)
    elif code.startswith("RETURN_"):
        # Return order → Different workflow
        handle_return_order(code)
    elif code.startswith("CHECK_"):
        # Quality check → No recording
        show_quality_checklist(code)
```

🧪 TESTING GUIDE

Manual Test Cases

Test 1: Basic Workflow

```
1. Start app
2. Login
3. Settings → Test Camera → ✅ OK
4. Settings → Start Camera → ✅ Started
5. Dashboard → Camera feed hiển thị
6. Quét QR → ✅ Detect
7. Auto record → ✅ Recording
8. Stop → ✅ Video saved
9. Storage → ✅ Video uploaded
```

Test 2: Error Handling

```
1. Start app (camera unplugged)
2. Settings → Test Camera → ❌ Error
3. Plug in camera
4. Test Camera → ✅ OK
5. Start Camera → ✅ Started
```

Test 3: S3 Upload

```
1. Settings → S3 Settings
2. Nhập thông tin S3
3. Test kết nối → ✅ OK
4. Lưu config
```

```
5. Record video
6. Storage → Upload history → ✅ Success
7. Storage → S3 tab → ✅ Video visible
```

Test 4: Cooldown

```
1. Quét QR "TEST123"
2. Auto record → Stop
3. Quét lại "TEST123" ngay
   → ❌ Console: "scanned recently. 298s cooldown remaining"
4. Đợi 5 phút
5. Quét lại "TEST123"
   → ✅ Auto record lại
```

📝 CODE STYLE

Python Style

```
# PEP 8 compliant
# Function names: snake_case
# Class names: PascalCase
# Constants: UPPER_CASE

# Type hints
def process_frame(frame: np.ndarray) -> Optional[str]:
    pass

# Docstrings
def function_name():
    """
    Brief description.

    Args:
        param1: Description

    Returns:
        Description
    """
    pass
```

Threading Best Practices

```
# Use locks for shared state
with state_lock:
    app_state["is_recording"] = True
```

```
# Use queues for inter-thread communication
frame = _frame_queue.get(timeout=0.01)

# Use daemon threads for background workers
thread = threading.Thread(target=worker, daemon=True)
thread.start()

# Use events for graceful shutdown
_stop_event.set()
_thread.join(timeout=2.0)
```

🔄 GIT WORKFLOW

Branch Strategy

```
main         Production-ready code
├── develop  Development branch
    ├── feature/camera-enhancement
    ├── feature/s3-retry
    └── bugfix/memory-leak
```

Commit Messages

```
feat: Add multi-camera support
fix: Fix RTSP buffer lag
perf: Optimize AI scanner FPS
docs: Update BUILD_GUIDE.md
refactor: Extract upload logic to service
```

📦 DEPENDENCIES

Core Dependencies

```
flask>=2.3.0              # Web framework
opencv-python             # Camera & video
zxing-cpp>=2.2.0          # QR/Barcode scanner
numpy                     # Array operations
```

```
boto3>=1.28.0             # S3 SDK
cryptography>=41.0.0      # Encryption
```

Build Dependencies

```
pyinstaller>=6.0.0        # EXE builder
```

Development Dependencies (Optional)

```
pytest                    # Testing
black                     # Code formatter
flake8                    # Linter
mypy                      # Type checker
```

🐛 KNOWN ISSUES & WORKAROUNDS

Issue 1: Memory leak in AI scanner

Symptom: App slows down over time

Cause: `_code_history` dict grows unbounded

Fix: Periodic cleanup (implemented)

```
# camera/ai_scanner.py (line ~115)
if current_time - last_cleanup_time > CLEANUP_INTERVAL:
    old_keys = [k for k, v in _code_history.items()
                if current_time - v > COOLDOWN_SECONDS]
    for k in old_keys:
        del _code_history[k]
```

Issue 2: RTSP lag 5 seconds

Symptom: Video lags behind real-time

Cause: RTSP buffer accumulation

Fix: Set buffer size to 0 (implemented)

```
# camera/camera_manager.py (line ~85)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 0)
```

Issue 3: Recording blocks camera

Symptom: Camera freezes during recording

Cause: Synchronous VideoWriter blocks thread

Fix: Async recorder with separate thread (implemented)

```
# camera/recorder.py
# Separate _write_loop() thread
# Non-blocking write_frame()
```

📚 REFERENCES

Flask

- Docs: https://flask.palletsprojects.com/
- Routing, Templates, Sessions

OpenCV

- Docs: https://docs.opencv.org/
- VideoCapture, VideoWriter, MJPEG

ZXing-CPP

- GitHub: https://github.com/zxing-cpp/zxing-cpp
- Faster than pyzbar

Boto3 (AWS SDK)

- Docs: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html
- S3 operations, Multipart upload

PyInstaller

- Docs: https://pyinstaller.org/
- .spec file, Hooks, Hidden imports

Inno Setup

- Docs: https://jrsoftware.org/ishelp/
- .iss script syntax

🔮 FUTURE ENHANCEMENTS

v1.1.0 Ideas

- [ ] Video compression (FFmpeg integration)
- [ ] Multiple S3 buckets
- [ ] Database integration (PostgreSQL)
- [ ] Real order API integration
- [ ] Webhook notifications
- [ ] Export statistics (Excel/CSV)
- [ ] Dark mode UI
- [ ] Mobile responsive design
- [ ] Docker deployment
- [ ] Kubernetes support

Version: 1.0.0
Updated: 09/02/2026
Team: EcoHub Development